MEMORANDUM

To:	Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From:	Gabriel Choi, Carmen Alaichamy
Date:	May 6, 2026
Subject:	Lab 3B: Closed-loop Velocity Control

Introduction

The objective of this lab was programming a PI controller using object-oriented programming and implement it on a microcontroller. This allowed us to test and evaluate the controller, testing the performance of the P-only, I-only, and PI controllers using step-response tests.

Methodology

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. We completed the class file controller.py shown in Appendix A to control velocity of the motor using the encoder information. We created a contructor __init__() to set the PI gains. Then methods set_setpoint() and set_gains() sets the setpoint and controller gains. Finally method run() takes in the current velocity, and returns the correct voltage the motor should run at. The class file was then uploaded to the microcontroller using Thonny.

A main program (main.py) attached in Appendix B was developed to perform a step response test. This file performs the closed-loop step response test.

The collected data was plotted using Python in a Jupyter Notebook, documented in Appendix C, showing the motor's step response and controller output with corresponding Kp and KI values. This allowed us to compare the effects of the proportional controller, integral controller, and both applied at the same time.

A Simulink model was created using the open-loop model, with the PI controller, DC model, and unity feedback. This was set to the optimal PI gains found in the last plot.

Hand calculations were also completed, calculating the theoretical time constant and steady-state values.

Results

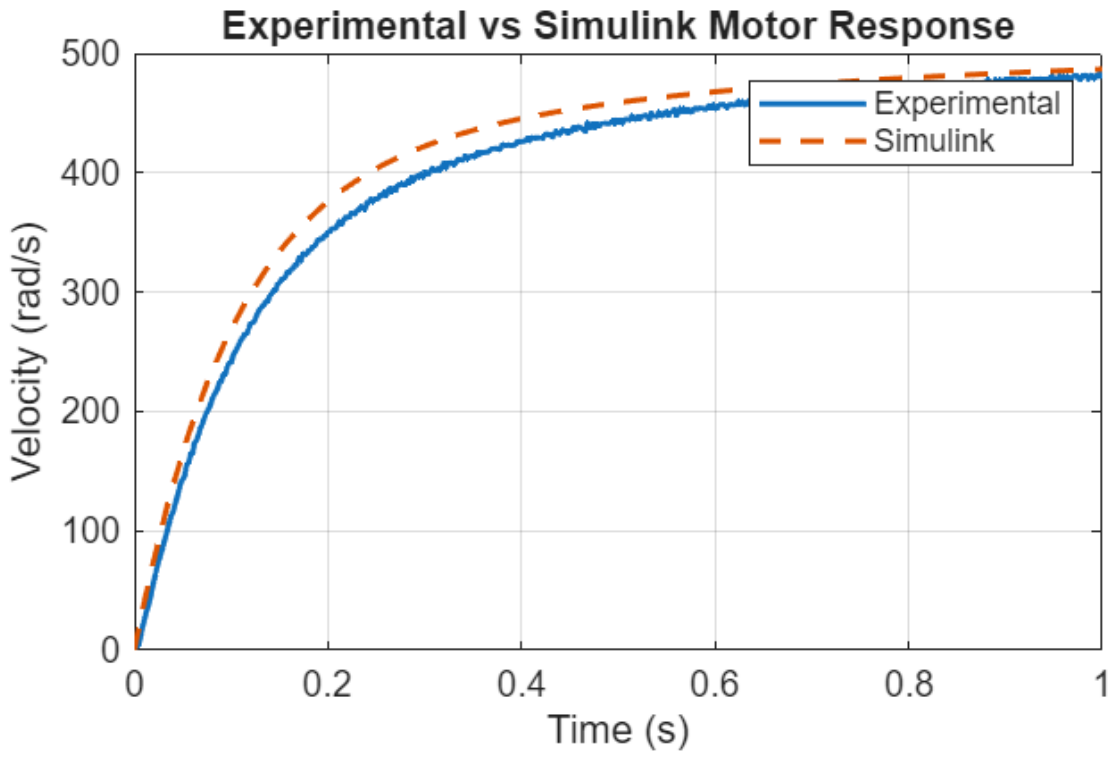


Figure 1. Optimal experimental and theoretical step response overlaid.

Comment on the differences between experimental and simulated step responses. What factors could contribute to the difference?

The simulated response closely follows the experimental motor speed response. Small differences are expected because the first-order model approximates the real motor dynamics and does not include all nonlinear effects such as friction, saturation, and measurement noise.

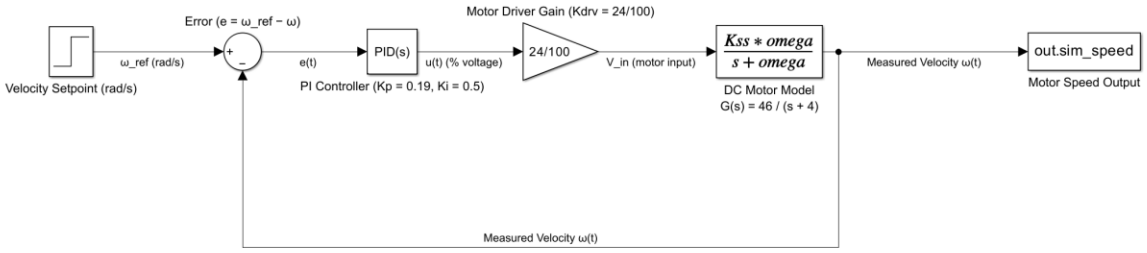


Figure 2. Theoretical Calculations

Compare theoretical calcs to experimental

Discussion

Discuss effects of proportional and integral gains on transient and steady-state responses.

The proportional and integral controllers serve different functions in affecting the step response. By testing proportional and integral controllers individually, then combined, we can see their different effects. Proportional controller makes systems faster, but the steady state response will still have error. While the integral controller decreases error, it takes a long time to reach steady state so the main portion of the response is transient.

This is proven with the plots in Appendix C, where in Figure C.2 the step response from only the proportional controller, where it shows that the plot reaches steady state quickly but has large steady state error. With only integral controller, in Figure C.3, the response takes too long to reach steady state however it is closer to the correct value. This is because the integral works over time.

How would you graphically determine whether the controller saturates?

Graphically, the controller saturates when the plot

In this experiment, why doesn't a large proportional gain alone produce oscillation, while a large integral gain does? Explain your answer using the location of closed loop poles with the PI controller and their corresponding time response.

In this experiment, a large proportional gain alone acts like a overdamped response, where Kp just moves towards the goal, however it often stops before reaching it, creating a steady state error. KI creates oscillation as the integral logs error over time, so it continues to corrects itself even as it passes the setpoint. The accumulated error causes the overshoot.